

Pautlen

Pautlen UAM | 2020 - 2021

Authors

Víctor Perea Riquelme | victor.perea@estudiante.uam.es

Álvaro Rodríguez Palacios | alvaro.rodriguezp@estudiante.uam.es

Javier Romera Llave | javier.romeral@estudiante.uam.es

Makefile

For help, run:

make help

all	Compile all.
clean	Cleans output files
default	Equivalent to 'make all'
exe	Executes with yyin as stdin & yyout as
stdout.	
help	This guide
test_all	Executes all tests
test_first	Executes first test
test_second	Executes second test
test_third	Executes third test

Comment about tokens processing

Token catch is kinda messy (for now!):

```
switch(token)
{
    case TOK_MAIN:
        /* PROCESS TOK_MAIN */
        ...
        ...
        ...
    default:
        break;
}
```

Folder structure

Important: shown tree before **make** and after **make clean**:

```
root
.
├── alfa
│   ├── alfa.c
│   └── alfa.l
├── inc
│   ├── alfa.h
│   └── tokens.h
├── Makefile
├── out
├── README.md
├── README.pdf
├── symbols
│   └── symbols.txt
├── testfiles
│   ├── entrada1.txt
│   ├── entrada2.txt
│   ├── entrada3.txt
│   ├── leeme.txt
│   ├── salida1.txt
│   ├── salida2.txt
│   └── salida3.txt
├── testresults
└── testverify.sh
```

Important: shown tree after **make**, **make test** and before **make clean**

```
root
.
├── alfa
│   ├── alfa.c
│   └── alfa.l
├── inc
│   ├── alfa.h
│   └── tokens.h
├── lex.yy.c
├── lex.yy.o
├── Makefile
├── out
│   └── pruebaMorfo
├── README.md
├── README.pdf
├── symbols
│   └── symbols.txt
├── testfiles
│   ├── entrada1.txt
│   ├── entrada2.txt
│   ├── entrada3.txt
│   ├── leeme.txt
│   ├── salida1.txt
│   ├── salida2.txt
│   └── salida3.txt
├── testresults
│   ├── testoutput1.txt
│   ├── testoutput2.txt
│   └── testoutput3.txt
└── testverify.sh
```

Tests

In order to help either us or whoever, we have provided a Makefile rule - `make test_all` - which will throw the result of all test.

Test can be tested separately:

```
make test_first
```

```
make test_second
```

```
make test_third
```

This rules execute **pruebaMorfo** with it's correspondant source testfile - located at *testfiles/* - and, helped by **testverify.sh**, a script which process the result of doing `diff -Bb inputfile.txt outputfile.txt`, shows a message with the result of the test. Expected return for diff is `\0`.

testverify.sh

```
#!/bin/bash

# info: checks if file_source is same to file_result
# args:
#   (0): script exec
#   (1): file_source
#   (2): file_result

ON_ERROR="TEST FAILURE."
ON_SUCCESS="TEST SUCCESS."

red='\e[1;31m'
grn='\e[1;32m'
end='\e[0m'

# Main

file_source=$1
file_result=$2

if [ $# -eq 2 ]; then
    printf 'Test Result: '
    DIFF=$(diff -Bb $file_source $file_result)
    if [ "$DIFF" != "" ]; then
        printf "${red}%s${end}\n" "$ON_ERROR"
    else
        printf "${grn}%s${end}\n" "$ON_SUCCESS"
    fi
else
    printf 'Bad args: %s FILE_SOURCE FILE_RESULT\n' "$0"
fi
```

For manual testing

If manual testing is deserved, you can type `make exe`. This will run **pruebaMorfo** with *stdin* and *stdout* as *yyin* and *yyout* respectively.

Alfa.c and Alfa.l

Functionality is managed by **alfa.c**, so **alfa.l** just passes the information processed. Thanks to this, preprocessor directives can be passed from Makefile - controlled text regions, see [gnu-gcc-preprocessor](#) -, which let us test either manually or automatically.

This is referred to this code (in **alfa.c**):

```
75  int setup(...)
76  {
    ...
81  #ifndef _STD_

    /* Select yyin and yyout from argv[1] and argv[2] */

    ...

103 #else

    yyin = stdin;
    yyout = stdout;

108 #endif

    ...

114 }
```

Common header file **alfa.h** contains `extern char *errbuff`, a common buffer to manage errors detected in **alfa.l** and handled at **alfa.c**.